ARCHITECTURE DECISION RECORD

Database Schema Design

HiddenGemMusic | Capstone Project | April 2026

Author: Leena | Status: Moved to Star Schema | Track: BDA

1. Context

This document records the architectural decisions made in designing the relational database schema for the GlobalGroove capstone project. GlobalGroove is a cross-regional music discovery platform that surfaces the Discovery Gap — globally trending songs and artists that have not yet crossed over into a user's local market.

The database is the foundation of the entire application. All overlap calculations, trend analysis, Hidden Gem logic, and chart comparisons are executed as stored procedures within the database and passed to the .NET 8 API layer as clean, aggregated results. Getting this schema right before any data is ingested or any stored procedures are written is critical.

2. Data Sources

The schema is designed to unify two Kaggle datasets that overlap in subject matter but differ significantly in structure, column naming, geographic encoding, and chart scope.

Dataset 1 — Spotify Historical Charts (2017–2021)

Approximately 3.5GB. Covers Spotify's Top 200 and Viral 50 charts published globally since January 1, 2017 through December 30, 2021. Charts are published every 2–3 days. Countries are written as full names (e.g. "United States"). Songs are identified by a Spotify URL from which a Spotify track ID can be parsed. Streams are NULL for all Viral 50 entries.

Source columns: title, rank, date, artist, url, region, chart, trend, streams

Dataset 2 — Top Spotify Songs in 73 Countries (2023–2025)

Approximately 500MB. Covers the Top 50 songs per country, updated daily from October 17, 2023 through June 10, 2025. Countries are represented as 2-letter ISO codes (e.g. "US"). Songs are identified by a native Spotify ID. Includes rich audio feature data (danceability, energy, tempo, valence, etc.) not present in Dataset 1.

Source columns: spotify_id, name, artists, daily_rank, daily_movement, weekly_movement, country, snapshot_date, popularity, is_explicit, duration_ms, album_name, album_release_date,

danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time_signature

Known Data Challenges

- The two datasets have a 22-month gap (December 2021 – October 2023) with no chart data. This is documented on the Welcome Screen and marked clearly in all time-series visualizations.
- Dataset 1 uses full country names; Dataset 2 uses ISO codes. The Country lookup table bridges this with both columns.
- Dataset 1 identifies songs by Spotify URL; Dataset 2 uses a native Spotify ID. The Spotify ID can be parsed from the Dataset 1 URL during cleaning, enabling cross-dataset song matching where possible.
- Dataset 1 covers Top 200 and Viral 50 charts; Dataset 2 covers only Top 50. Chart type is tracked via the ChartType lookup table.
- Popularity is measured differently across datasets: Dataset 1 uses raw stream counts; Dataset 2 uses Spotify's proprietary popularity score. Both are stored separately and never compared directly.
- Some rows in Dataset 2 have NULL country values, indicating Global Top 50 entries. These are handled by allowing NULL on the country_id FK in ChartEntry.

3. Architectural Decisions

Decision 1 — Full normalization to 3NF

The schema is fully normalized to Third Normal Form (3NF). Every non-key attribute depends only on the primary key of its table, with no transitive dependencies. Lookup data (countries, chart types) lives in dedicated tables rather than being stored as repeated strings across millions of chart entry rows.

Rationale: At 13–16 million estimated rows, storing repeated strings like country names or chart type labels inline would waste significant storage and make updates inconsistent. Normalization keeps the data clean, consistent, and efficient for the types of aggregate queries this application requires.

Decision 2 — Single ChartEntry table for all chart data

All chart entries — regardless of dataset origin, chart type, or country — are stored in a single ChartEntry table. Chart type is distinguished via a FK to the ChartType lookup table. Dataset-specific fields that do not exist in both datasets (streams, popularity, daily_movement, weekly_movement, trend) are stored as nullable columns rather than in separate tables.

Rationale: Splitting chart entries by dataset or chart type would create parallel tables with near-identical structures, requiring UNION queries for any cross-dataset analysis — which is the

entire point of this application. A single table with nullable columns is the cleaner and more query-friendly approach.

Decision 3 — Audio features in a separate AudioFeatures table

Audio feature columns (danceability, energy, tempo, valence, etc.) are stored in a dedicated AudioFeatures table with a one-to-one FK relationship to Song, rather than as nullable columns on the Song table.

Rationale: Audio features are a cohesive, optional group of 13 columns that exist only for Dataset 2 songs. They are not required for any MVP feature — the core Discovery Gap analysis, overlap calculations, and Hidden Gem logic operate entirely on chart rank data. Separating them signals clearly in the schema that these are secondary, reach-goal data. It also keeps the Song table focused on song identity, and allows audio feature queries to be written and optimized independently of core chart queries.

The one-to-one relationship is enforced by a UNIQUE constraint on song_id in AudioFeatures.

Decision 4 — Many-to-many Artist-Song relationship via ArtistSong

Songs and artists are related through a join table (ArtistSong) rather than storing artist data as a string on the Song table. A boolean is_primary column distinguishes the primary credited artist from featured artists.

Rationale: Dataset 2 stores artists as a comma-separated string (e.g. "Drake, 21 Savage") and explicitly instructs consumers to split on ', ' to get a list. Storing this as a string would make artist-level filtering, grouping, and aggregation impossible without string parsing in every query. Normalizing artists into their own table with a join enables clean artist-level stored procedures and future artist profile features.

Decision 5 — Weekly movement stored on ChartEntry, not in a separate table

Daily movement and weekly movement are stored as nullable columns on ChartEntry rather than in a separate movement or trend table.

Rationale: Movement data is a property of a specific chart entry on a specific date — it is not a separate entity. Dataset 2 provides these values pre-calculated; Dataset 1 does not include them. Storing them as nullable columns on ChartEntry is the simplest and most direct approach. A separate table would add joins without adding clarity or resolving any normalization concern, since movement is functionally dependent on the chart entry itself.

Decision 6 — Both Viral 50 and Top 200 included from Dataset 1

Both the Top 200 and Viral 50 charts from Dataset 1 are ingested and stored. Each is represented as a distinct ChartType row.

Rationale: The Viral 50 tracks songs gaining rapid momentum — shares, playlist adds, and growing buzz — often before a song appears on the Top 200. This is directly aligned with the Discovery Gap concept: a song may be going viral in one region before it becomes mainstream in another. Excluding the Viral 50 would remove potentially the most signal-rich data for identifying early-stage cross-regional trends.

4. Schema Reference

All tables are defined below with column names, data types, constraints, and descriptions. Primary keys use IDENTITY(1,1) for auto-increment. All foreign keys enforce referential integrity. Nullable columns are explicitly marked.

Country

Lookup table bridging Dataset 1 full country names with Dataset 2 ISO codes. Used as a FK in ChartEntry to standardize geographic references across both datasets.

Column	Type	Constraints	Description
`country_id`	`INT`	PK, IDENTITY	Auto-incremented surrogate primary key
`full_name`	`NVARCHAR(100)`	NOT NULL, UNIQUE	Full country name as used in Dataset 1 (e.g. 'United States')
`iso_code`	`CHAR(2)`	NOT NULL, UNIQUE	2-letter ISO 3166-1 alpha-2 code as used in Dataset 2 (e.g. 'US')

ChartType

Lookup table defining the three chart types present across both datasets: Top 200, Viral 50, and Top 50. Allows ChartEntry rows to be filtered and grouped by chart type without storing repeated strings.

Column	Type	Constraints	Description
`chart_type_id`	`INT`	PK, IDENTITY	Auto-incremented surrogate primary key
`name`	`NVARCHAR(50)`	NOT NULL, UNIQUE	Chart name: 'Top 200', 'Viral 50', or 'Top 50'

Artist

Stores individual artist entities. Each unique artist name gets exactly one row. The ArtistSong join table handles the many-to-many relationship between artists and songs, including featured artist credits.

Column	Type	Constraints	Description
artist_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
name	NVARCHAR(255)	NOT NULL, UNIQUE	Artist display name as it appears in the source datasets

Album

Stores album metadata from Dataset 2. Albums are an optional relationship — Dataset 1 contains no album data, so album_id is nullable on the Song table. Included to support potential future audio feature and mood comparison functionality.

Column	Type	Constraints	Description
album_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
name	NVARCHAR(255)	NOT NULL	Album title as provided in Dataset 2
release_date	DATE	NULL	Album release date from Dataset 2. Nullable as format may vary or be missing.

Song

The core song identity table. Each unique track gets exactly one row, regardless of how many countries it charts in or how many chart entries reference it. The spotify_id column enables cross-dataset song matching where possible — parsed from the URL in Dataset 1 and stored natively from Dataset 2.

Column	Type	Constraints	Description
song_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
spotify_id	NVARCHAR(50)	NULL, UNIQUE	Spotify track identifier. Parsed from URL in Dataset 1; native field in Dataset 2. Nullable for rows where parsing fails or is ambiguous.
title	NVARCHAR(255)	NOT NULL	Song title as it appears in the source dataset
duration_ms	INT	NULL	Song duration in milliseconds. Available in Dataset 2 only.

Column	Type	Constraints	Description
is_explicit	BIT	NULL	Whether the track contains explicit lyrics. Available in Dataset 2 only.
album_id	INT	NULL, FK -> Album	Reference to the album this song belongs to. Nullable — no album data in Dataset 1.

ArtistSong

Join table resolving the many-to-many relationship between Artist and Song. Every artist credited on a track gets a row here. The is_primary flag distinguishes the main credited artist from featured artists, which is important for artist-level queries and display logic in the Vinyl card component.

Column	Type	Constraints	Description
artist_song_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
artist_id	INT	NOT NULL, FK -> Artist	Reference to the credited artist
song_id	INT	NOT NULL, FK -> Song	Reference to the song
is_primary	BIT	NOT NULL, DEFAULT 1	True if this is the primary credited artist. False for featured artists. Used to distinguish 'Drake ft. 21 Savage' from '21 Savage ft. Drake'.

AudioFeatures

Stores Spotify's audio analysis data for songs from Dataset 2. This table is intentionally separated from Song to signal that audio features are secondary, reach-goal data not required for any MVP feature. The UNIQUE constraint on song_id enforces the strict one-to-one relationship — a song either has audio features or it doesn't.

All feature columns are nullable to accommodate any rows where Spotify's API returned incomplete data during Dataset 2 collection.

Column	Type	Constraints	Description
audio_features_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
song_id	INT	NOT NULL, UNIQUE, FK -> Song	Reference to the song. UNIQUE enforces one-to-one relationship.

Column	Type	Constraints	Description
danceability	FLOAT	NULL	How suitable the track is for dancing. Range 0.0–1.0.
energy	FLOAT	NULL	Perceptual measure of intensity and activity. Range 0.0–1.0.
key	TINYINT	NULL	Musical key using standard Pitch Class notation (0 = C, 1 = C#/Db, ... 11 = B). -1 if no key detected.
loudness	FLOAT	NULL	Overall loudness in decibels (dB). Typical range -60 to 0 dB.
mode	TINYINT	NULL	Modality of the track. 1 = major, 0 = minor.
speechiness	FLOAT	NULL	Presence of spoken words. Values above 0.66 are likely speech-only. Range 0.0–1.0.
acousticness	FLOAT	NULL	Confidence measure of whether the track is acoustic. Range 0.0–1.0.
instrumentalness	FLOAT	NULL	Predicts whether a track contains no vocals. Values above 0.5 are intended as instrumental. Range 0.0–1.0.
liveness	FLOAT	NULL	Detects presence of a live audience. Values above 0.8 indicate a strong likelihood of live performance. Range 0.0–1.0.
valence	FLOAT	NULL	Musical positiveness conveyed by the track. High valence = happy/cheerful. Low valence = sad/angry. Range 0.0–1.0.
tempo	FLOAT	NULL	Estimated tempo in beats per minute (BPM).
time_signature	TINYINT	NULL	Estimated overall time signature. Specifies how many beats are in each bar. Range 3–7.

ChartEntry

The largest and most central table in the schema. Every row represents a single song's appearance on a specific chart, in a specific country, on a specific date. This is the table that all stored procedures query to calculate overlap percentages, trend velocity, Hidden Gem scores, and global distribution metrics.

Estimated row count: 13–16 million rows across both datasets combined. All heavy computation against this table is executed server-side in stored procedures, with only aggregated results passed to the API layer.

Column	Type	Constraints	Description
chart_entry_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key

Column	Type	Constraints	Description
song_id	INT	NOT NULL, FK -> Song	Reference to the charting song
country_id	INT	NULL, FK -> Country	Reference to the country. Nullable to accommodate Dataset 2 Global Top 50 entries where country is NULL.
chart_type_id	INT	NOT NULL, FK -> ChartType	Reference to the chart type (Top 200, Viral 50, or Top 50)
snapshot_date	DATE	NOT NULL	The date this chart entry was recorded. Maps to 'date' in Dataset 1 and 'snapshot_date' in Dataset 2.
rank	SMALLINT	NOT NULL	The song's rank position on this chart on this date. Range 1–200 depending on chart type.
streams	BIGINT	NULL	Raw stream count. Available in Dataset 1 Top 200 entries only. NULL for all Viral 50 and Dataset 2 entries.
popularity	TINYINT	NULL	Spotify's proprietary popularity score (0–100). Available in Dataset 2 only. Not comparable to stream counts — never used alongside streams in the same calculation.
daily_movement	SMALLINT	NULL	Change in rank compared to the previous day. Available in Dataset 2 only.
weekly_movement	SMALLINT	NULL	Change in rank compared to the same day the previous week. Available in Dataset 2 only.
trend	NVARCHAR(20)	NULL	Trend direction string from Dataset 1 ('MOVE_UP', 'MOVE_DOWN', 'SAME', 'NEW_ENTRY'). Not present in Dataset 2.

5. Entity Relationships Summary

The following relationships are enforced at the database level via foreign key constraints:

- Song -> Album: Many-to-one. Many songs can belong to one album. Nullable — Dataset 1 has no album data.
- ArtistSong -> Artist: Many-to-one. Many ArtistSong rows can reference one artist.
- ArtistSong -> Song: Many-to-one. Many ArtistSong rows can reference one song. Together, ArtistSong resolves the many-to-many between Artist and Song.
- AudioFeatures -> Song: One-to-one. Enforced by UNIQUE constraint on song_id in AudioFeatures.
- ChartEntry -> Song: Many-to-one. One song can have thousands of chart entries across countries and dates.
- ChartEntry -> Country: Many-to-one. Nullable to support Global Top 50 entries.
- ChartEntry -> ChartType: Many-to-one. One chart type applies to many chart entries.

6. Data Cleaning Notes

The following cleaning steps are required before ingestion and are tracked in the accompanying data cleaning summary document:

- Dataset 1: Parse Spotify track ID from the URL column and populate spotify_id on Song.
- Dataset 1: Map full country names to iso_code using the Country lookup table.
- Dataset 2: Split the artists column on ', ' to produce individual Artist rows and ArtistSong join rows.
- Dataset 2: Identify and handle NULL country rows as Global Top 50 entries (country_id = NULL on ChartEntry).
- Both datasets: Deduplicate Song rows where the same track appears under slightly different title strings. Use spotify_id as the primary deduplication key where available.
- Both datasets: Validate rank ranges against chart type (e.g. rank > 50 should not exist for Top 50 entries).
- Both datasets: Confirm snapshot_date formats and normalize to DATE before ingestion.

7. Consequences and Trade-offs

Accepting this schema design entails the following known trade-offs:

- The 22-month data gap (December 2021 – October 2023) cannot be resolved by schema design. It is a data limitation that must be communicated transparently in the UI and accounted for in all time-series stored procedures.
- Cross-dataset song matching via spotify_id will not be 100% complete. Some Dataset 1 URLs may fail to parse cleanly. Songs that cannot be matched will exist as separate Song rows for each dataset.
- Streams and popularity scores are stored in separate columns and must never be used in the same comparative calculation. Stored procedures must be written to account for which metric is available for a given chart entry.
- Audio feature data is only available for Dataset 2 songs. Any stored procedures using AudioFeatures must LEFT JOIN and handle NULLs appropriately.
- The ChartEntry table at 13–16 million rows will be the primary performance concern. All stored procedures must use appropriate indexing strategies — at minimum on (country_id, snapshot_date) and (song_id, country_id) — to avoid full table scans.

8. Decision Status

Status: Proposed — pending team review at the Week 2 design check-in meeting.

This ADR will be updated to Accepted once the team has reviewed and agreed on the schema at the design check-in. Any changes agreed upon in that meeting will be reflected in a revised version of this document committed to the repository.

Authored by Leena | GlobalGroove Capstone | April 2026